



Catlike Coding › Unity › Hex Map

published 2026-02-27

Hex Map 5.2.0 Save and Load Menu



The new save/load menu.

This tutorial is made with Unity 6000.3.8f1 and follows Hex Map 5.1.0.

1 UI Document

Last time we migrated the New Map menu to UI Toolkit and now we're taking care of the Save and Load menu. We'll use the same approach as before. The Save and Load menu is a bit more complex, because it has two modes, has an input field, and has to show a list of names.

We can create its UI document by duplicating *New Map Panel.uxml* and naming it *Save Load Panel.uxml*. Remove all child elements of the overlay panel and change its width to 300px.

```
<?xml version="1.0" encoding="utf-8"?>
<ui:UXML
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:ui="UnityEngine.UIElements"
  xmlns:uie="UnityEditor.UIElements"
  xsi:noNamespaceSchemaLocation="../../UIElementsSchema/UIElements.xsd">
  <Style src="UI Styles.uss"/>
  <ui:VisualElement class="overlayBackground">
    <ui:VisualElement class="overlayPanel" style="width: 300px;">
      //...
    </ui:VisualElement>
  </ui:VisualElement>
</ui:UXML>
```

This panel also has a centered label, but its text depends on whether the save or load menu is shown. So give it a name to identify it with. Let's just use *Save* for its placeholder text.

```
<ui:VisualElement class="overlayPanel" style="width: 300px;">
  <ui:Label text="Save" name="MenuLabel" style="align-self: center;"/>
</ui:VisualElement>
```

The panel has a row with three buttons at its bottom. We can create that using the same structure that we used in the right side panel. The first button is a variable action button, and we'll again use *Save* for its placeholder text.

```
<ui:Label text="Save" name="MenuLabel" style="align-self: center;"/>
<ui:VisualElement style="flex-direction: row;">
  <ui:Button text="Save" name="ActionButton"/>
  <ui:Button text="Delete" name="DeleteButton"/>
  <ui:Button text="Cancel" name="CancelButton"/>
</ui:VisualElement>
```

Above the buttons sits the text field that will hold the target map name. We use a **TextField** element for this, with its name set to *NameField*.

```
<ui:Label text="Save" name="MenuLabel" style="align-self: center;"/>
<ui:TextField name="NameField"/>
<ui:VisualElement style="flex-direction: row;">
  ...
</ui:VisualElement>
```

The field is supposed to display *Enter or select a map name...* when it is empty. We can configure that via its `placeholder-text` attribute.

```
<ui:TextField
  name="NameField"
  placeholder-text="Enter or select a map name..."/>
```

To make the placeholder text disappear when the text field receives focus set its `hide-placeholder-on-focus` attribute to `true`.

```
<ui:TextField
  name="NameField"
  placeholder-text="Enter or select a map name..."
  hide-placeholder-on-focus="true"/>
```

The placeholder text of the old menu is italic. We can make the new one italic as well by giving it the `-unity-font-style: italic` style. Its style class is `.unity-base-text-field__input--placeholder`. Add it to *UI Styles.uss*.

```
.unity-base-text-field__input--placeholder {
  -unity-font-style: italic;
}
```

The last element that we have to add to *Save Load Panel.uxml* is something to show the stored maps, for which we'll use a `ListView`. It sits between the label and text field. Give it the *MapList* name.

```
<ui:Label text="Save" name="MenuLabel" style="align-self: center;"/>
<ui:ListView name="MapList"/>
<ui:TextField .../>
```

The default and simplest way to make the list view lay out its contents is to give all its items the same height, expressed in pixels. We configure this height via its `fixed-item-height` attribute. We use the standard label height, which is 20.

```
<ui:ListView name="MapList" fixed-item-height="20"/>
```

Let's also style the list a bit. Give it a 4px margin so it better fits with the other contents of the panel. Also, the list automatically grows its as much as it can to show its items. We will limit this by giving it a maximum height. Set its **max-height** to 200px. This will let the list grow to show up to ten items. If there are more items then it will show a scrollbar.

```
<ui:ListView
    name="MapList"
    fixed-item-height="20"
    style="margin: 4px; max-height: 200px;"/>
```

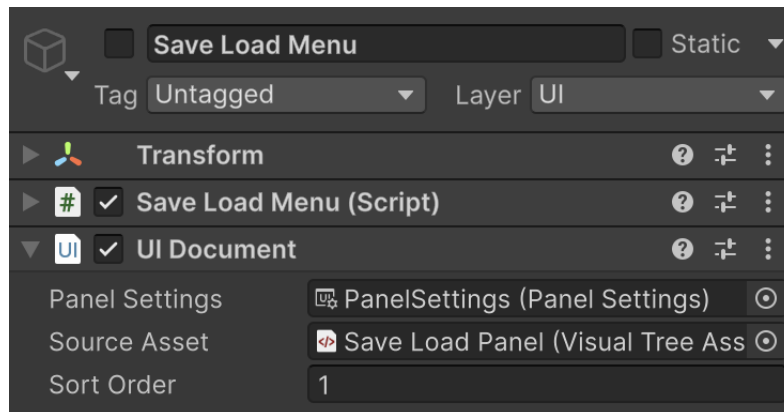
We can now view our panel in the UI Builder. It will show that our empty list takes up vertical space for one item.



Menu in UI Builder.

2 Game Object

Add a **UIDocument** component to the *UI / Save Load Menu* game object and configure it like the New Map menu, but with the appropriate document asset. Then remove the components for the old UI and the child objects, and revert the **Transform** component, like we did before. We again have to remove the components in the correct order to avoid complaints about the required components.



Save Load Menu components.

By temporarily activating the game object we can see the new menu in the game, but will always be in its default state and won't list any maps.

3 Component

What's left is to adjust **SaveLoadMenu**. Begin by replacing the usage of the `UnityEngine.UI` namespace with `UnityEngine.UIElements`. Removal of the old namespace will cause some errors to appear, but we'll get rid of that code shortly.

```
using UnityEngine;
// using UnityEngine.UI;
using UnityEngine.UIElements;
using System;
using System.IO;
```

Can we now remove the uGUI package?

No, even though we no longer make use of it directly, the Scriptable Render Pipeline Core package still has it as a dependency, which is used by URP.

Like we did for the New Map menu, we mark **UIDocument** as required component. Then we remove the old serialized fields, only keeping the reference to **HexGrid**.

```
[RequireComponent(typeof(UIDocument))]
public class SaveLoadMenu : MonoBehaviour
{
    const int mapFileVersion = 5;

    // [SerializeField]
    // Text menuLabel, actionButtonLabel;

    // [SerializeField]
    // InputField nameInput;

    // [SerializeField]
    // RectTransform listContent;

    // [SerializeField]
    // SaveLoadItem itemPrefab;
```

We instead add fields for the **TextField** and **ListView** UI elements. These don't need to be serializable because we'll query for them when we open the menu.

```
bool saveMode;

TextField nameField;

ListView mapList;
```

Remove all code from `Open`, only keeping the assignment of the save mode, game object activation, and locking of the camera.

```

public void Open(bool saveMode)
{
    this.saveMode = saveMode;
    // if (saveMode) { ... }
    // else { ... }
    // FillList();
    gameObject.SetActive(true);
    HexMapCamera.Locked = true;
}

```

The `SelectItem` method can be removed. It was only used by the old list item objects.

```

// public void SelectItem(string name) => nameInput.text = name;

```

In `Delete` we now have to clear the `value` property of the name field.

```

public void Delete()
{
    ...
    nameField.value = "";
    FillList();
}

```

And in `GetSelectedPath` we retrieve the same property.

```

string GetSelectedPath()
{
    string mapName = nameField.value;
    ...
}

```

Next, we modify `FillList` to get rid of the old game objects. Remove the loops that destroy and create game objects for the old list items. From now on we only have to keep track of the map names, for which we introduce a field to replace the local variable. Then we immediately convert the full paths to only the map file names without extension and sort them.

```

string[] mapNames;

...

void FillList()
{
    // for (int i = 0; i < listContent.childCount; i++) { ... }
    mapNames = Directory.GetFiles(Application.persistentDataPath, "*.map");
    for (int i = 0; i < mapNames.Length; i++)
    {
        mapNames[i] = Path.GetFileNameWithoutExtension(mapNames[i]);
    }
    Array.Sort(mapNames);
    // for (int i = 0; i < paths.Length; i++) { ... }
}

```

Now we're going to set up the UI document instance. We again do this in `Open` and begin by retrieving the root element.

```

public void Open(bool saveMode)
{
    this.saveMode = saveMode;
    gameObject.SetActive(true);
    HexMapCamera.Locked = true;

    VisualElement root = GetComponent<UIDocument>().rootVisualElement;
}

```

Let's set the menu label first. Retrieve the `Label` and set its text appropriately.

```

VisualElement root = GetComponent<UIDocument>().rootVisualElement;

root.Q<Label>("MenuLabel").text = saveMode ? "Save Map" : "Load Map";

```

Then hook up the buttons to their respective methods, and set the text of the action button.

```

root.Q<Label>("MenuLabel").text = saveMode ? "Save Map" : "Load Map";

var actionButton = root.Q<Button>("ActionButton");
actionButton.text = saveMode ? "Save" : "Load";
actionButton.clicked += Action;

root.Q<Button>("DeleteButton").clicked += Delete;
root.Q<Button>("CancelButton").clicked += Close;

```

After that get a reference to the `TextField`.

```

root.Q<Button>("CancelButton").clicked += Close;

nameField = root.Q<TextField>("NameField");

```


Do the same for the `ListView`.

```
nameField = root.Q<TextField>("NameField");  
mapList = root.Q<ListView>("MapList");
```

To make the map list functional we have to set up a few things. First, we have to tell it how to make list items. This could be done in UXML via templates, but we only use a single label per item so templates are overkill. We instead provide a delegate for the list's `makeItem` property with code to construct an item. It simply returns a new `Label`.

```
mapList = root.Q<ListView>("MapList");  
mapList.makeItem = static () => new Label();
```

The list creates enough item instances to fill itself, then reuses those visual elements to show the items that are currently in view. When it has to show an item it will invoke a delegate, passing it a `VisualElement` and item index. We have to provide this delegate via its `bindItem` property. We have to cast the element to a `Label` and set its text to the appropriate map name.

```
mapList.makeItem = static () => new Label();  
mapList.bindItem = (e, i) => ((Label)e).text = mapNames[i];
```

We also have to respond when an item of the list has been selected. It has multiple events that we could use for that. We'll use the `selectedIndicesChanged` event. This provides us with a list of selection indices, because the list could be configured to allow the selection of multiple items at the same time. We don't support that, so instead of working with the indices we can just use the list's `selectedIndex` to set the name field's text to the correct name.

```
mapList.bindItem = (e, i) => ((Label)e).text = mapNames[i];  
mapList.selectedIndicesChanged += (indices) =>  
    nameField.value = mapNames[mapList.selectedIndex];
```

Why not just use the first selected index?

The indices are provided via an `IEnumerable`, from which we cannot directly pick an index. We could invoke `First` or `Single` on it, but that makes us depend on `System.Linq`, which is overkill for this simple case.

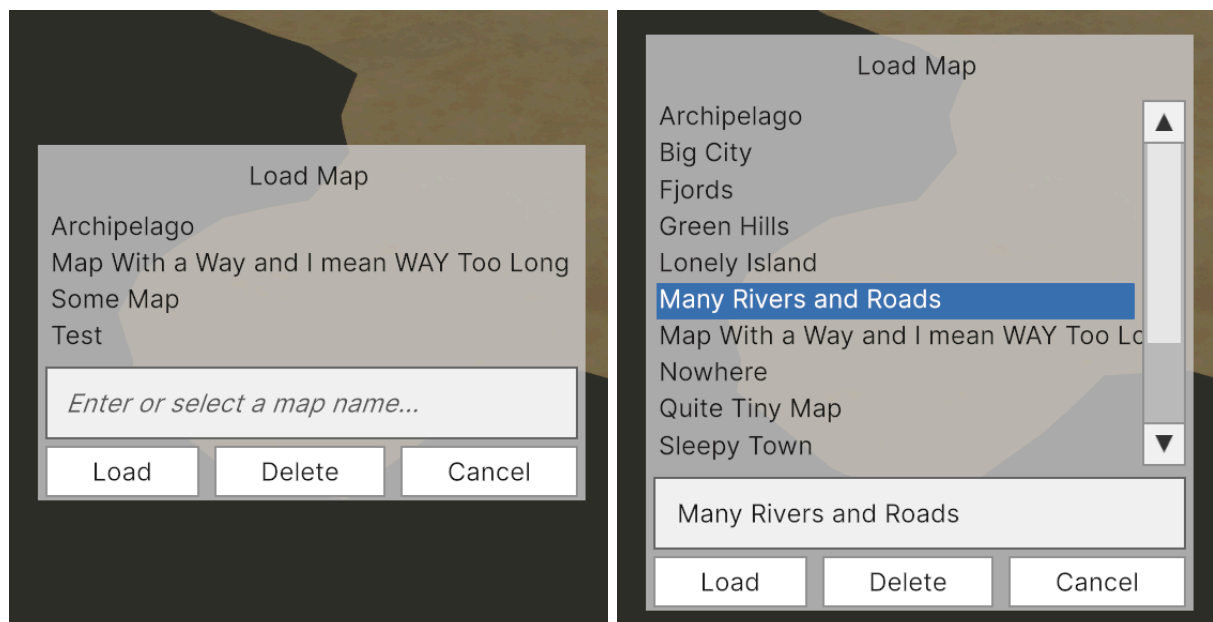
After all this has been set up we invoke `FillList` to get the map names.

```
mapList.selectedIndicesChanged += (indices) =>
    nameField.value = (string)mapList.selectedItem;

FillList();
```

And to finally make the list display those names we have to assign the map names to its `itemSource` property. Do this in `FillList`, so the list also gets updated when we delete a map.

```
void FillList()
{
    ...
    mapList.itemSource = mapNames;
}
```



Load menu with four and with fourteen maps.

Could we make the list always have the same height?

Yes, by settings its **height** style instead of **max-height**.

We finish by removing the `SaveLoadItem` script asset, as it is no longer needed.

This takes care of the Save/Load menu and the conversion to UI Toolkit is finished. In the future we'll get back to the in-game map itself.

Enjoying the tutorials? Are they useful?

Please support me on Patreon or Ko-fi!

Or make a direct donation!

made by Jasper Flick